LifeLine - A Hospital Integrated Pharmacy Management System

Submitted By

STUDENT NAME	STUDENT ID
ARFANUL HAQUE	232-15-**157**
TANZIA BINTE ROUF	232-15-**217**
RIPA AKTER	232-15-**417**
TOWFIQUE HASSAN	232-15-**955**

LifeLine Lab Project Report

This Report Presented in Partial Fulfillment of the course **CSE324: Operating Systems Lab** under **Computer Science and Engineering (CSE)** Department.

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 18, 2026

DECLARATION

We hereby declare that this **LifeLine** Lab Project has been done by us under the supervision of **Hefzul Hossain Papon**, **Lecturer**, Department of **Computer Science and Engineering**, **Daffodil International University**. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To

_____

HEFZUL HOSSAIN PAPON

Lecturer

Department of **Computer Science and Engineering**

Daffodil International University

Submitted By

TANZIA BINTE ROUF Student ID: **232-15-217** Dept. of CSE, DIU	**TOWFIQUE HASSAN** Student ID: **232-15-955** Dept. of CSE, DIU
RIPA AKTER Student ID: **232-15-417** Dept. of CSE, DIU	**ARFANUL HAQUE** Student ID: **232-15-157** Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course has course outcomes as follows:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Demonstrate the ability to use Linux shell environments to perform high-level data manipulation and system-level automation for healthcare applications.
CO2	Apply core Operating System principles, including process control, user authentication, and file system permissions, to create a secure management environment.
CO3	Evaluate system security and data integrity by implementing role-based access control and encrypted-lite authentication protocols in a Bash environment.
CO4	Construct a fully functional, GUI-supported Pharmacy Management System (LifeLine-RxGuard) that integrates inventory control, billing, and audit logging.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP2	EP1
CO2	PO2	C3	KP3	EP2
CO3	PO3	C4	KP4	EP3
CO4	PO3	C6	KP5	EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**

Table of Contents

Declaration i

Course & Program Outcome ii

1 Introduction 1
1.1 Introduction 2
1.2 Motivation 3
1.3 Objectives 3
1.4 Feasibility Study 3
1.5 Gap Analysis 4
1.6 Project Outcome 5

2 Proposed Methodology/Architecture 6
2.1 Requirement Analysis & Design Specification 6
2.1.1 Overview 6
2.1.2 Proposed Architecture/ System Design 6
2.1.3 UI Design 8
2.2 Overall Project Plan 10

3 Implementation and Results 12
3.1 Implementation 12
3.2 Sample Code & Processing Flow 13
3.3 Performance Analysis 28
3.4 Results and Discussion 29

4 Engineering Standards and Mapping 30
4.1 Impact on Society, Environment and Sustainability 30
4.1.1 Impact on Life 30
4.1.2 Impact on Society & Environment 30
4.1.3 Ethical Aspects 30
4.1.4 Sustainability Plan 30
4.2 Project Management and Team Work 31
4.3 Complex Engineering Problem 33
4.3.1 Mapping of Program Outcome 34
4.3.2 Complex Problem Solving 34
4.3.3 Complex Engineering Activities 35

5 Conclusion 36
5.1 Summary 36
5.2 Limitation 38
5.3 Future Work 39

References 40

Chapter 1

Introduction

1.1 Introduction

LifeLine-RxGuard is a Linux-based **Hospital Integrated Pharmacy Management System** developed using **Bash shell scripting with Zenity support**. The project is designed to provide a lightweight, secure, and efficient local-host application for managing important pharmacy operations within a healthcare environment.

Healthcare institutions frequently require organized systems to manage medicine inventory, patient billing, blood bank records, donor information, staff access control, and restricted medicine authorization. In many small-scale healthcare environments, these activities are often handled manually or through disconnected systems, which may lead to record inconsistency, operational delays, and security risks.

Linux operating systems provide a stable environment for automation and local application development. By using Bash shell scripting together with Zenity graphical dialogs, LifeLine-RxGuard transforms Linux shell capabilities into a practical application that behaves like a lightweight local software solution.

A key feature of this project is its **Zenity-based graphical interaction model**. Zenity allows Bash scripts to display graphical forms, menu selections, alerts, input dialogs, and confirmation windows. As a result, users do not need to manually type complex commands; instead, they interact through simple graphical menus.

The project works as a modular healthcare application where different pharmacy-related operations are grouped into functional sections. Users first authenticate through role-based login, and based on their assigned role, they gain access to different operational modules.

The system mainly supports two user roles:

- Pharmacist
- Staff

The pharmacist can perform advanced administrative tasks such as medicine registration, restricted medicine approval, user creation, blood inventory management, billing generation, backup creation, and audit monitoring.

Staff members can perform limited operational tasks such as medicine viewing, medicine issuing, donor handling, and request generation.

The project demonstrates how Bash shell scripting can be extended beyond command automation to create a structured healthcare management application that solves real operational problems efficiently within a Linux environment.

1.2 Motivation

The development of LifeLine-RxGuard was motivated by several practical problems commonly observed in pharmacy and healthcare management environments.

In many small healthcare institutions, pharmacy operations are still maintained manually. Medicine records are often stored in paper files or simple spreadsheets, which creates problems such as:

- Difficulty tracking medicine availability
- Delayed billing processes
- Poor restricted medicine control
- Incomplete donor information management
- Weak accountability in medicine distribution

One major concern in healthcare management is restricted medicine handling. Certain medicines require authorization before distribution, and without proper control there is a risk of misuse, unauthorized issue, or incorrect medicine release.

Another major motivation was to create a lightweight healthcare solution that does not depend on:

- Heavy database servers
- Web-based deployment
- Continuous internet connectivity
- Expensive enterprise software

Instead of complex commercial healthcare systems, the goal was to create a practical Linux-native solution using only built-in shell tools and flat-file storage.

The project was also academically motivated to demonstrate practical application of:

- Linux shell scripting
- File processing
- User authentication
- Role-based access control
- Healthcare workflow automation

This project therefore serves both practical and educational purposes.

1.3 Objectives

The LifeLine-RxGuard project was designed with a specific set of clinical and technical objectives:

- **Lightweight Hospital Integrated PMS:** To develop a lightweight hospital integrated pharmacy management system using Bash shell scripting, enabling efficient management without heavy software dependencies.
- **Comprehensive Inventory Management:** To create a system that provides real-time visibility into medicine stock levels, unit prices, and categories.
- **Security & Role Enforcement:** To implement a strict Role-Based Access Control (RBAC) system where "Pharmacists" have administrative rights and "Staff" members have restricted operational rights.
- **Blood Resource Tracking:** To maintain a live database of blood groups and donor eligibility to assist in medical emergencies.
- **Restricted Medicine Authorization:** To provide restricted medicine authorization requiring pharmacist approval, preventing unauthorized dispensing of critical medicines
- **Blood Inventory Records:** To maintain blood inventory records, tracking availability, usage, and expiry of different blood types
- **Financial Accuracy:** To eliminate manual calculation errors by automating the billing process using high-precision shell arithmetic.
- **Audit Accountability:** To implement an immutable audit trail that logs every user action, ensuring that no transaction goes untracked.
- **Backup Support:** To provide backup support for critical records, ensuring data safety in case of accidental loss or system failure.
- **User Accessibility:** To provide a professional Zenity based GUI experience that lowers the barrier to entry for non-technical hospital staff.

1.4 Feasibility Study

The feasibility of the LifeLine-RxGuard project is strong because it uses simple technologies that are widely available and easy to implement.

- **Technical Feasibility**

The project is technically feasible because it uses Linux-native technologies that are already available in standard Linux distributions.

Required technologies include:

- Bash Shell
- Zenity
- grep
- awk
- sed
- cut
- echo
- file handling commands

These tools allow the entire system to operate without external frameworks.

- **Economic Feasibility**

The project requires no commercial software. The project uses 100% open-source technologies. There are no licensing fees, and because it runs on minimal hardware (even an old Pentium-class PC), the total cost of ownership for a hospital is zero.

Required resources are:

- Linux operating system
- Zenity package
- Terminal environment

No licensing cost is involved.

- **Operational Feasibility**

The menu-driven graphical interface makes the system practical for real operational use.

Users can perform tasks without deep Linux knowledge. The use of simple dialogs and clear menu options ensures that staff members can be trained in under thirty minutes.

1.5 Gap Analysis

Several limitations exist in existing manual pharmacy systems:

- Manual medicine records are error-prone.
- Restricted medicine approval is difficult to track.
- Blood stock data is often unorganized.
- Donor information may be incomplete.

- Commercial systems are expensive.
- Manual logs lack searchability and offer no security.
- LifeLine-RxGuard is a local-first, offline-always solution.
- Provides ERP-like searchability and data organization.
- Implements cryptographic-lite hashing for secure user authentication.
- Automates billing and inventory management to reduce errors.
- Maintains audit logs for accountability.
- Supports backup of critical records to prevent data loss.
- Offers a simple, Zenity-based graphical interface for easy use.

Existing Problem	Limitation
Manual medicine records	High risk of errors
Paper billing	Slow processing
Separate donor records	Data inconsistency
Commercial pharmacy systems	High cost

LifeLine-RxGuard addresses these gaps through integrated local automation.

1.6 Project Outcome

The final outcome of the LifeLine-RxGuard project is a functional Bash-based and Zenity-supported healthcare application that performs multiple pharmacy management tasks efficiently.

The system allows users to:

- Manage medicine inventory
- View medicine details
- Issue medicines
- Handle restricted medicine requests
- Generate bills
- Maintain blood inventory
- Register donors
- Manage users
- Create backups
- Maintain audit logs

The project successfully demonstrates practical use of shell scripting in healthcare management.

Chapter 2

Proposed Methodology / Architecture

2.1 Requirement Analysis & Design Specification

Developing an efficient **hospital-integrated pharmacy management system** requires clear **functional and non-functional requirements**. **LifeLine-RxGuard** was designed as a **lightweight shell-based healthcare application**, integrating multiple pharmacy operations through Bash scripting and a Zenity-based graphical interface.

The requirement analysis focuses on identifying the necessary **software tools, hardware resources, and system features** required to implement the project successfully.

2.1.1 Overview

LifeLine-RxGuard is designed to operate fully within a **Linux environment**, requiring only Bash and Zenity for execution. The system is implemented as a **single modular Bash script file** named **LifeLine.sh**, where each "Management Pillar" functions as a separate module:

- **Inventory Management:** Handling medicines, stock updates, and expiry tracking.
- **Billing Module:** Automating patient billing and invoice generation.
- **Blood Bank Management:** Tracking blood stock levels and managing donor information.
- **User Management:** Role-based access control with secure login and audit logging.

The system follows a **Modular Scripting Design**, separating the **business logic** from the **presentation layer**:

- **Frontend:** Zenity-driven menus, forms, and dialogs for interactive user experience.
- **Backend:** Bash scripting functions utilizing **flat-file storage** (text files) for inventory, bills, donors, and logs.

The modular approach allows **independent development, testing, and maintenance** of each functional unit while maintaining integration with the main system.

2.1.2 Proposed Architecture (Client–System Model)

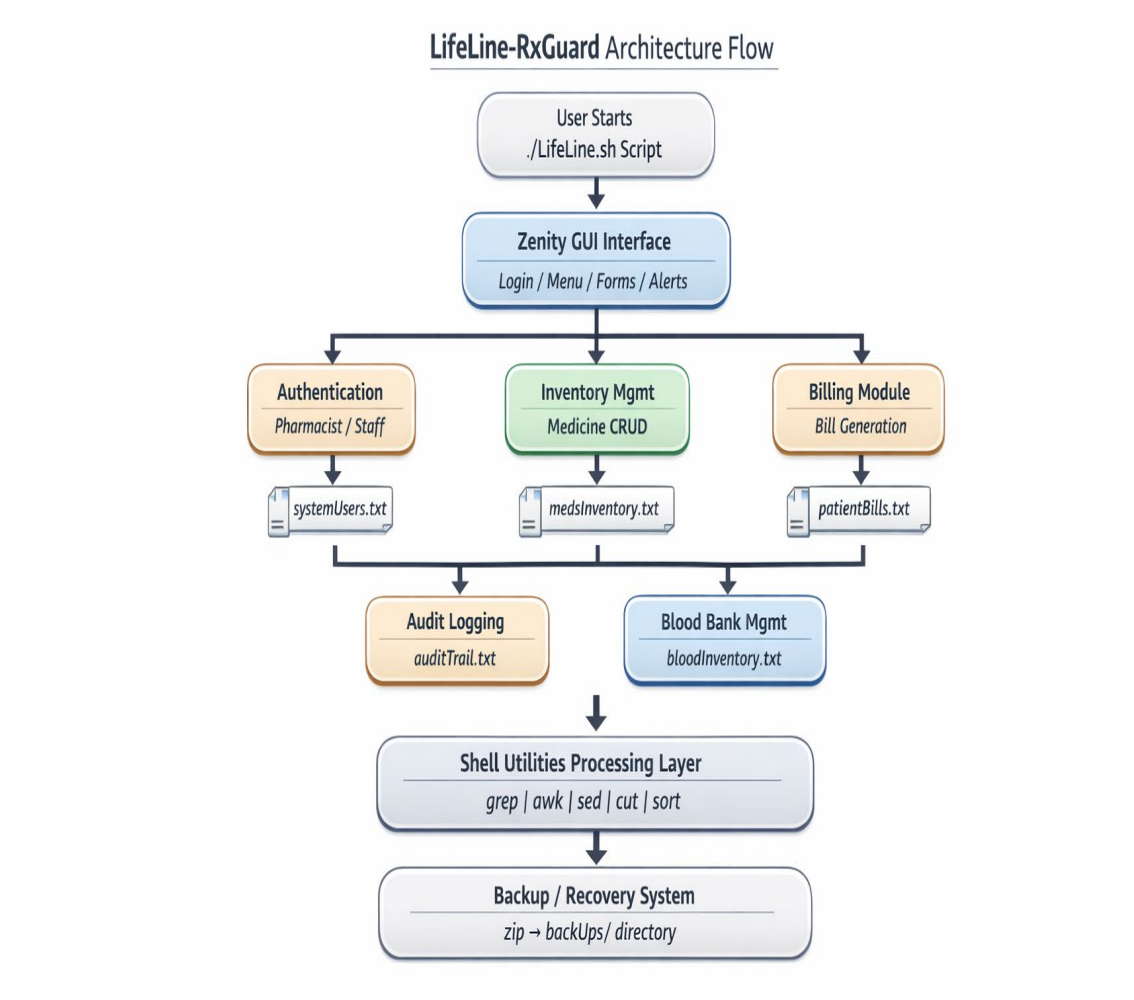
The LifeLine-RxGuard project follows a simple architecture where the user interacts with the shell script through the terminal. The script processes the input and executes Linux commands to retrieve system information.

The system follows a **Modular Client-System Model**. When the script is executed, it spawns a background process that checks for the existence of the database files. If they are missing, it triggers an **Initialization Wizard**.

Data Architecture:

- Uses **flat-file storage** for each module, e.g., medsInventory.txt, patientBills.txt, bloodStock.txt, and donorRecords.txt.
- Implements **pseudo-relational linking**, e.g., Medicine ID in patientBills.txt corresponds to entries in medsInventory.txt.
- Supports **easy backups** and modular updates without complex database engines.

Architecture Flow



In this architecture:

- The user executes **LifeLine.sh** from the Linux terminal.
- A **login prompt** is displayed for role-based authentication (Pharmacist/Staff/Admin).
- Based on the role, a **role-specific menu** is shown using Zenity.

- The user selects operations such as inventory updates, billing, blood management, or donor registration.
- The script updates the respective **database files** and records activities in the **audit log**.
- Users can **log out safely**, and administrators can export **backup files**.

This design ensures the system remains **lightweight, efficient, and maintainable** while providing **full pharmacy management functionality**.

2.1.3 UI Design

LifeLine-RxGuard is a command-line tool, the user interface is designed to be simple and user-friendly. The interface uses a **menu-based approach**, which allows users to select multiple options easily.

Key UI Components and Modules

1. Secure Authentication Portal (The Gateway)
Upon execution of lifeLine.sh, the user is presented with a centered Zenity login dialog. To prevent "shoulder surfing" in a busy hospital environment, the password field is masked using the --hide-text flag. The UI logic here is critical: if the user cancels the login or enters incorrect credentials, the script terminates immediately, preventing any "backdoor" access to the medicine database.

2. The Pharmacist Dashboard (Admin UI)
Once authenticated as a Pharmacist, the interface opens a "Master Control" menu. This menu is designed for high-level oversight and includes:

- User Management UI: Graphical forms to provision new Staff accounts or revoke access.
- Restricted Medicine Approval: A specialized alert window that pops up if a Staff member has requested a restricted substance (e.g., opioids or high-schedule drugs).
- Audit Log Viewer: A read-only, scrollable text window that displays the entire history of system actions, categorized by timestamp and User ID.
- Inventory Modification: Tools to edit existing medicine prices or delete discontinued items.

3. The Staff Dashboard (Operational UI)
The Staff menu is streamlined for clinical efficiency, focusing on rapid data entry and retrieval. Key components include:

- Point-of-Sale (POS) Billing: A step-by-step UI where the staff selects a medicine from a dropdown list, enters the quantity, and receives an instant visual total.

- Blood Donor Registry: Simple forms to register new donors and a color-coded list view to check available blood units (e.g., A+, B- inventory).
- Medicine Search: A real-time search bar that filters the medicine inventory and displays stock levels in a table format.

4. Data Validation and Feedback Dialogs
To prevent "Bad Data" from corrupting the flat-file database, the UI includes several feedback mechanisms:

- Progress Bars: During database backups or large inventory loads, a Zenity progress bar informs the user of the system's status.
- Error Alerts: If a staff member enters a quantity greater than what is available in stock, a "Warning" dialog appears with an "Insufficient Stock" message.
- Confirmation Popups: Before any medicine is deleted or a bill is finalized, a "Question" dialog asks the user to confirm the action, reducing the risk of accidental data loss.

5. Immutable Audit Logging (Back-end UI)
While not visible as a constant menu, every UI interaction triggers a background "Logging Event." For example, when the UI displays "Bill Generated Successfully," the backend simultaneously writes a non-erasable line to the auditTrail.txt. This ensures that every mouse click and form submission is accounted for in the system's history.

6. Session Exit and File Safety
The "Logout" option in the UI performs a "Safe Exit." It clears temporary variables, closes any open file descriptors, and returns the terminal to its original state, ensuring that the next person to use the computer cannot hijack the previous session.

Menu Display

The LifeLine.sh script presents a **menu-driven Dashboard** where users can select different system functions using the terminal with Zenity popups.

The **menu is organized clearly** with command numbers and descriptions, improving usability.

User Input

- Users interact with the system by **clicking or typing** the corresponding menu option.
- Zenity provides **graphical forms, lists, and input dialogs**, making the system easy to use even for staff with minimal technical knowledge.

Output Display

- **Results are displayed in Zenity popup windows or the terminal**, depending on the function.
- Example outputs: lists of medicines, blood bank stock levels, donor information, billing details, and system status.
- The organized display ensures users can **quickly access and understand the information** without clutter.

2.2 Overall Project Plan

The LifeLine-RxGuard project was developed through a series of structured phases to ensure systematic design, implementation, testing, and documentation. Each phase was carefully planned to guide the project from conceptualization to final deployment.

The development process followed a clear sequence so that each module could be completed in an organized and manageable way. Regular review at each stage helped maintain consistency and ensured that project goals were achieved successfully.

Phase 1: Research and Planning (Week 1–2)

- Conducted research on **Bash shell scripting, Zenity GUI design, and flat-file database management**.
- Identified **project requirements**, including:
  - Medicine inventory management
  - Billing automation
  - Blood inventory tracking
  - Donor registration
  - Audit logging and restricted medicine authorization
- Planned the **modular script architecture**, dividing the system into logical components (inventory, billing, blood bank, donor records, audit logging).
- Designed the **user interaction model** with a menu-driven Zenity interface to ensure usability for hospital staff.

Phase 2: Script Development (Week 3–4)

- Developed the **individual Bash scripts** for each module:
  - **Medicine Inventory Module** – Add, search, update, and delete medicine records.
  - **Billing Module** – Generate bills automatically for pharmacy transactions.
  - **Blood Bank Module** – Track blood stock and manage donor information.
  - **Audit Module** – Record all user activities for accountability.
- Implemented **role-based access control** for different user levels (pharmacist, staff, admin).
- Each function was tested individually to ensure correct operation and accurate output.

Phase 3: Integration (Week 5–6)

- Combined all module scripts into a single file: **LifeLine.sh**.
- Implemented a **Zenity-based graphical interface**, allowing users to select operations through menus rather than command-line inputs.
- Verified that **all modules worked together**, ensuring seamless interaction between inventory, billing, donor records, and audit logs.

Phase 4: Testing and Debugging (Week 7–8)

- Conducted extensive testing in Linux environments to ensure **script functionality and stability**.
- Performed debugging tasks, including:
    - Correcting Zenity menu navigation issues
    - Ensuring accurate inventory updates and billing calculations
    - Handling invalid user inputs
    - Verifying audit logging for all user activities
- Repeated executions confirmed **reliable performance under different scenarios**, including large medicine and donor records.

Phase 5: Documentation (Week 9)

- Prepared comprehensive **project documentation**, including:
    - System design diagrams and workflow charts
    - Module explanations and code structure
    - Testing results and performance analysis
    - Future scope and sustainability impact
- Finalized the **project report** with all supporting materials, ensuring clarity for evaluators and stakeholders.

Chapter 3

Implementation and Results

3.1 Implementation

The implementation of **LifeLine-RxGuard** was carried out using **Bash shell scripting with Zenity support**. Bash is a powerful scripting language commonly used for automating system administration tasks in Linux environments, while **Zenity** provides graphical dialog boxes for better user interaction.

The main script file **LifeLine.sh** contains multiple functions that perform different system monitoring tasks.

The implementation involves the following steps:

- Creating a login-based authentication system
- Designing a role-based menu-driven interface
- Defining operational modules for pharmacy tasks
- Executing file-based record processing
- Displaying outputs through Zenity dialogs

Each function in the script is responsible for executing a specific healthcare operation and updating records accordingly.

For example:

- Medicine registration
- Medicine issue
- Memory-safe file updates
- Blood inventory handling
- Billing generation
- Donor registration
- Restricted medicine approval

The script uses:

- Conditional statements
- Loops
- File validation
- User input checking

to control execution safely.

3.2 Sample Code & Processing Flow

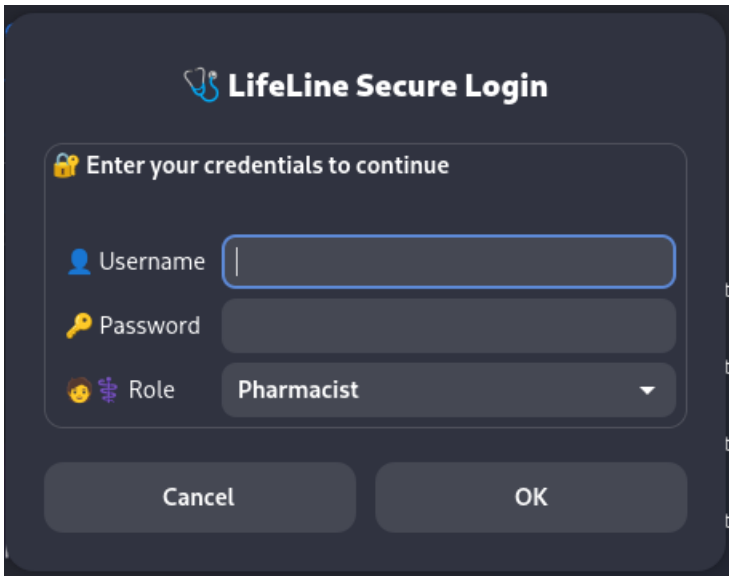
The LifeLine-RxGuard script executes different Linux commands depending on the user's selection.

Example Execution Flow:

Step 1: User runs the script

bash LifeLine.sh

Step 2: The system displays a menu



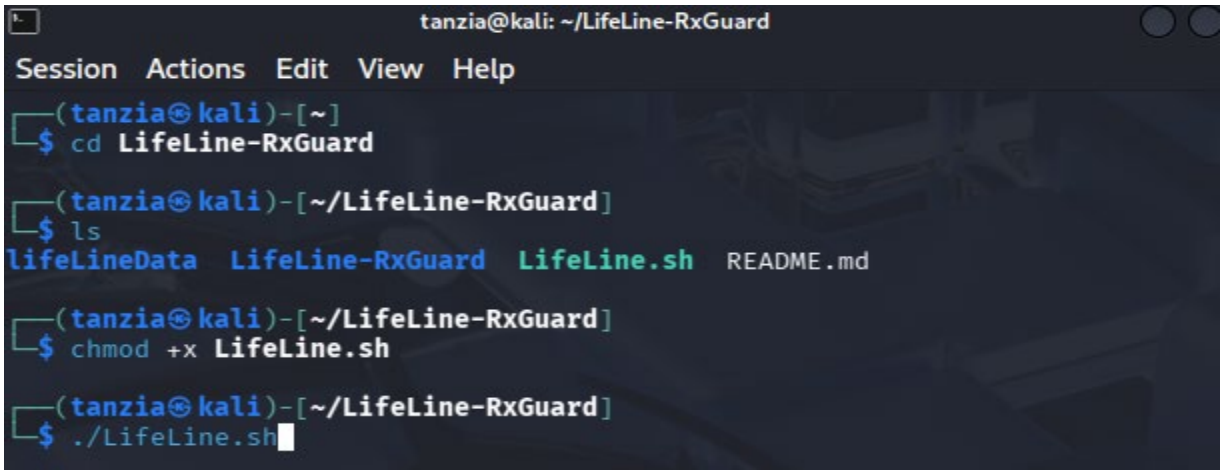
Step 3: User puts username & password. Then selects an option of role.

Step 4: The script executes the corresponding command based on the role.

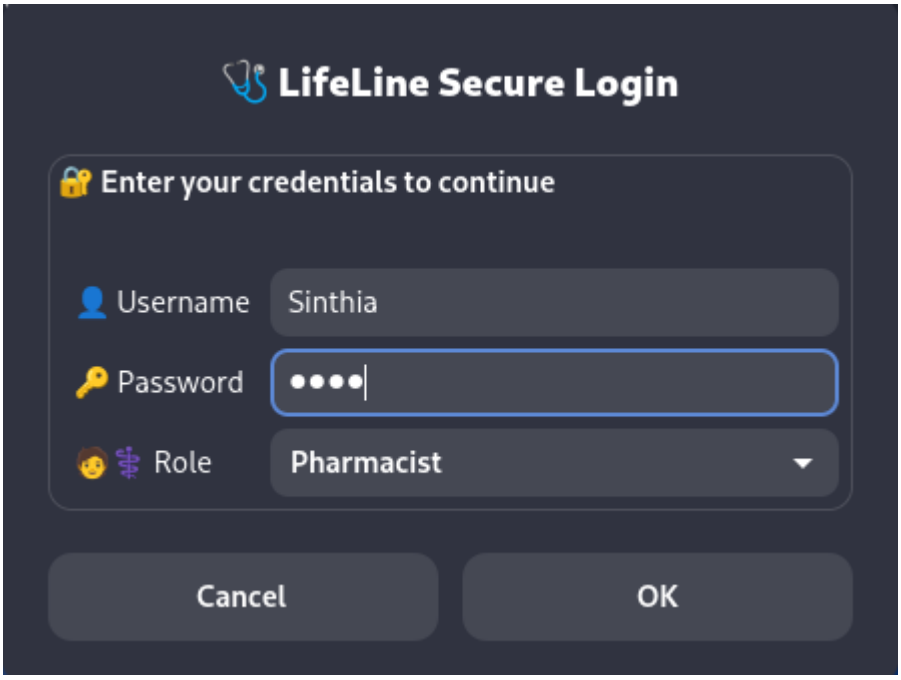
LifeLine Code Processing Flow:

LifeLine Images:

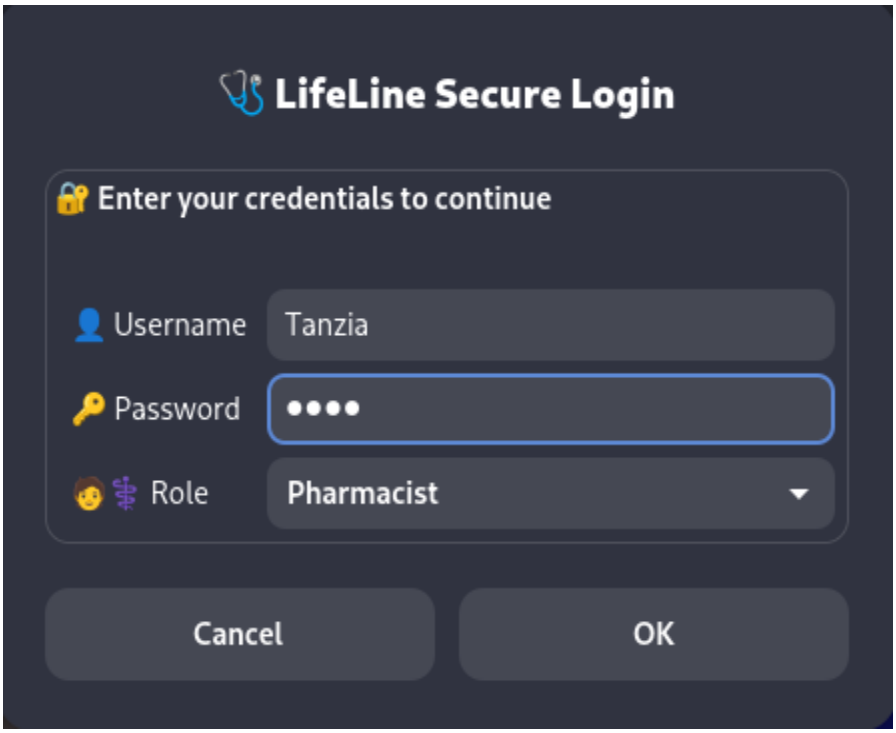
- **CLI for LifeLine:**

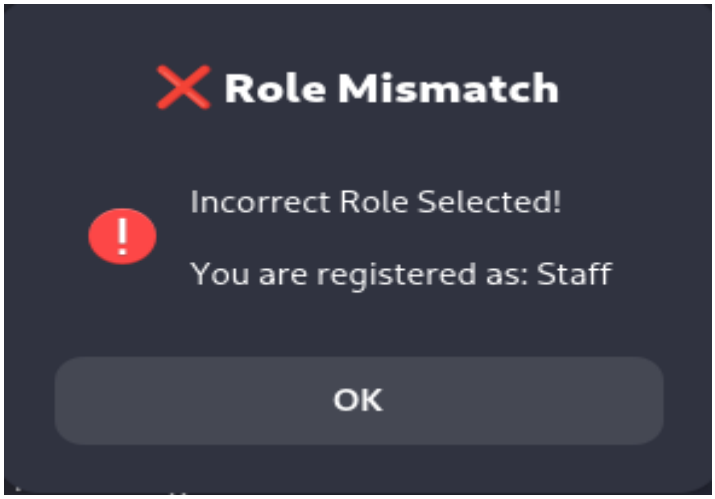


- **Login Screen:**

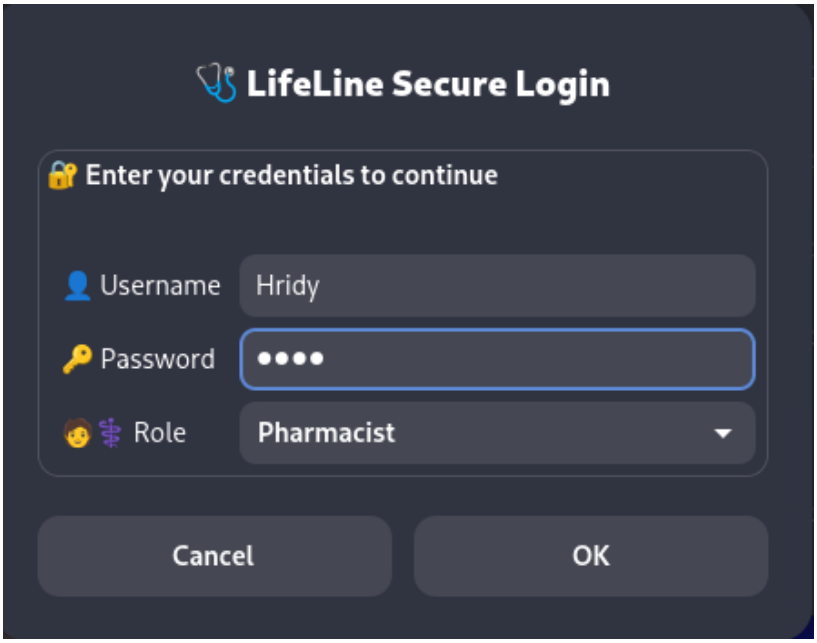


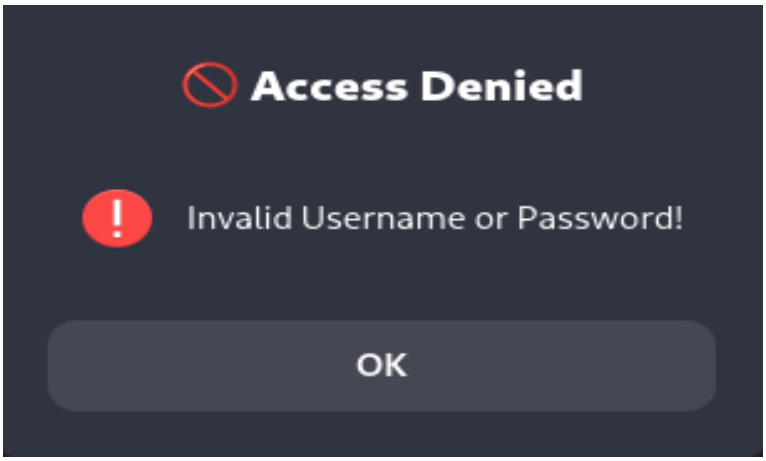
- **Incorrect Login Credentials:**



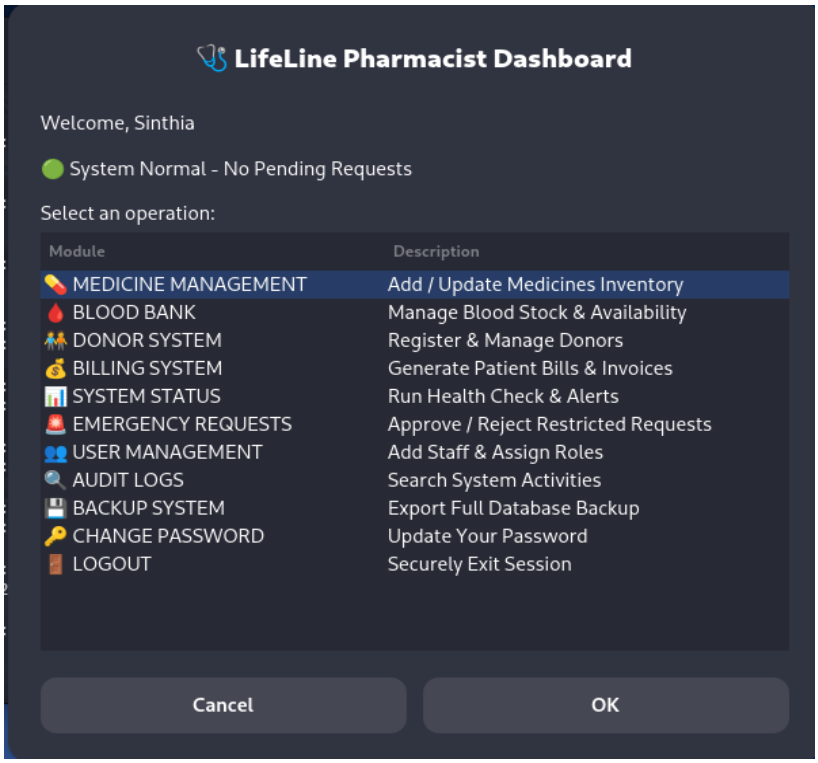


- **Invalid Login Credentials:**

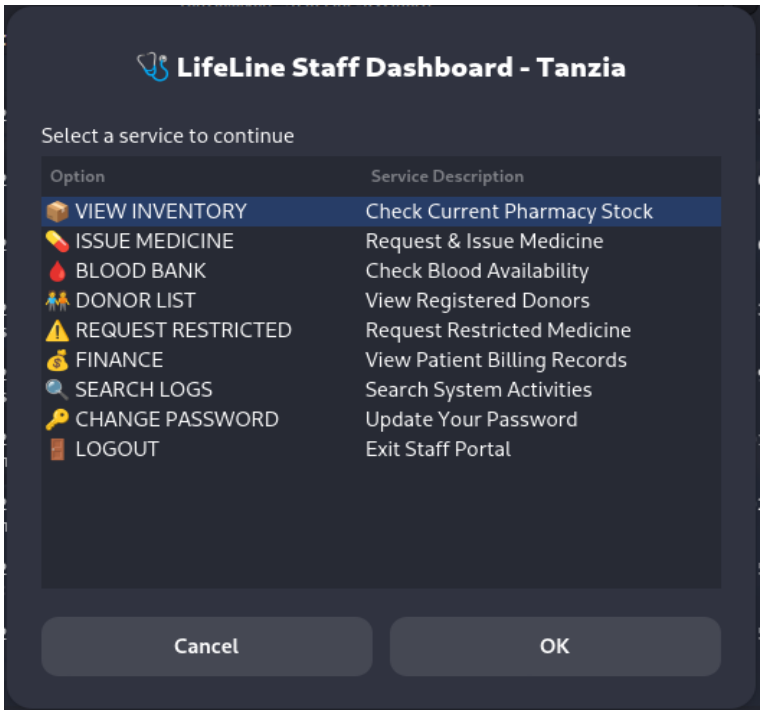




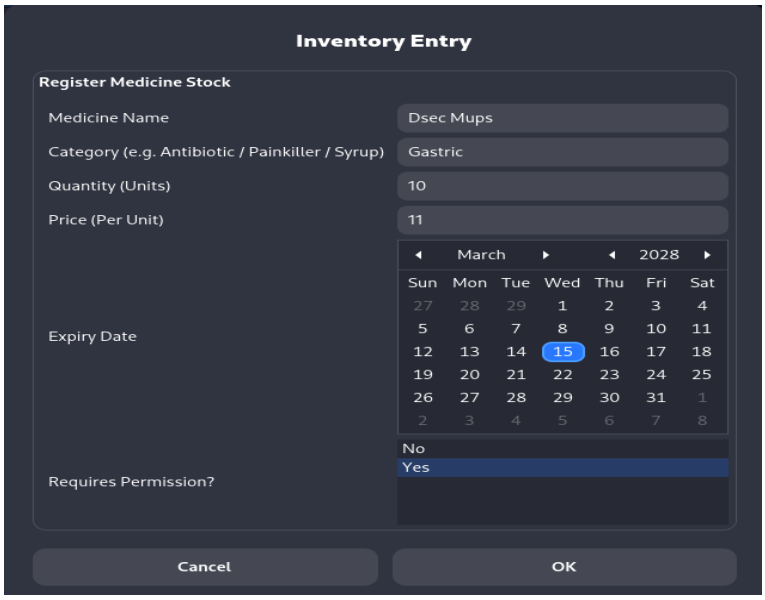
- **Features:**
  - **Pharmacist:**

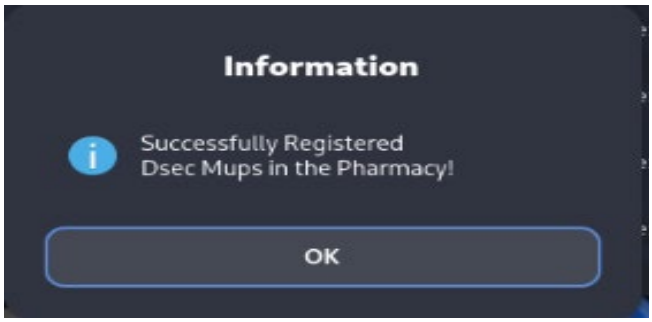


  - **Staff:**

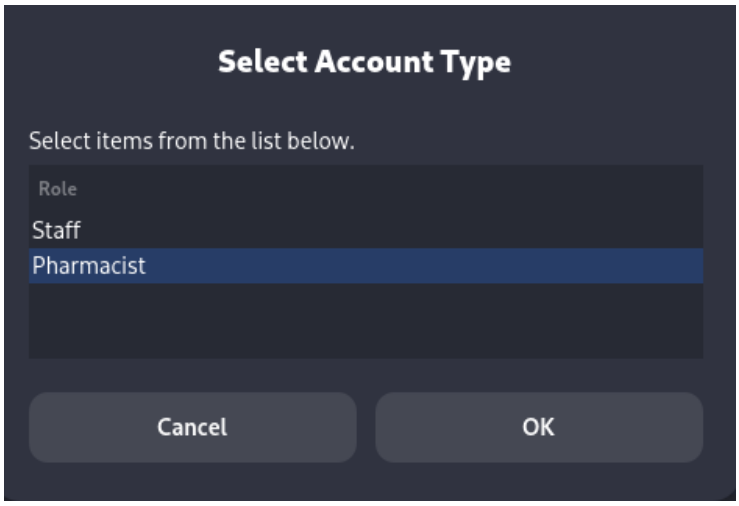


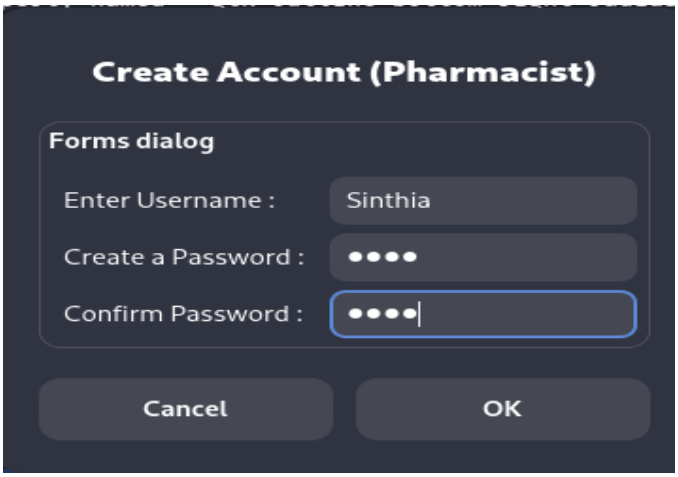
- **Add Medicine:**

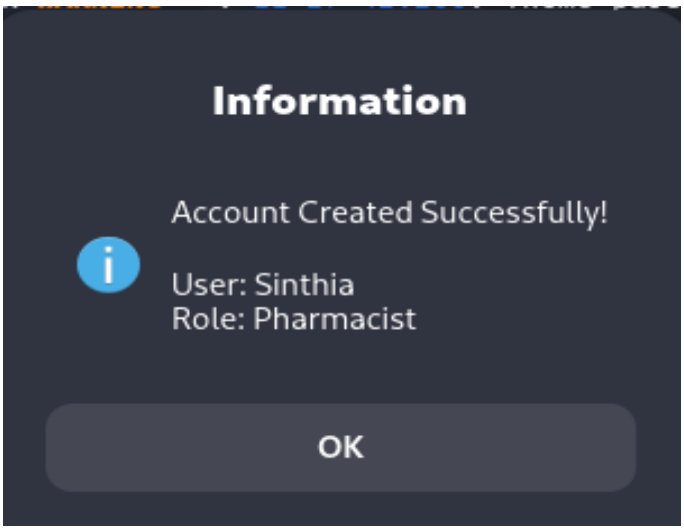




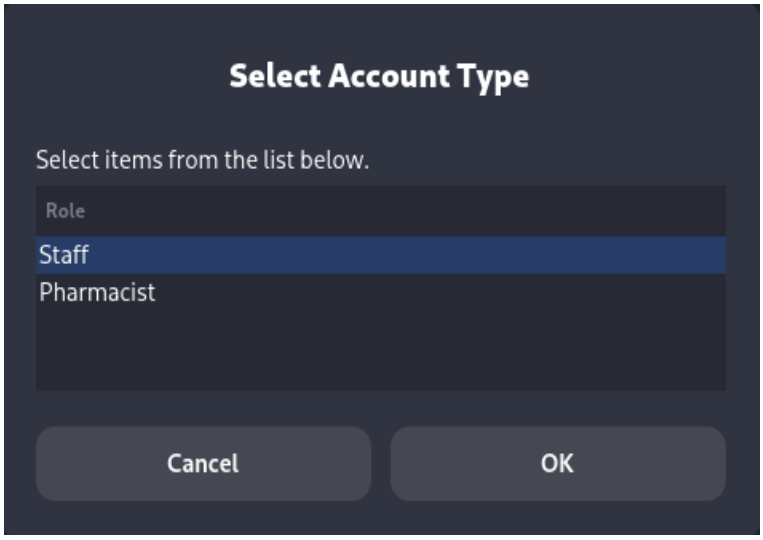
- **Add Pharmacist:**

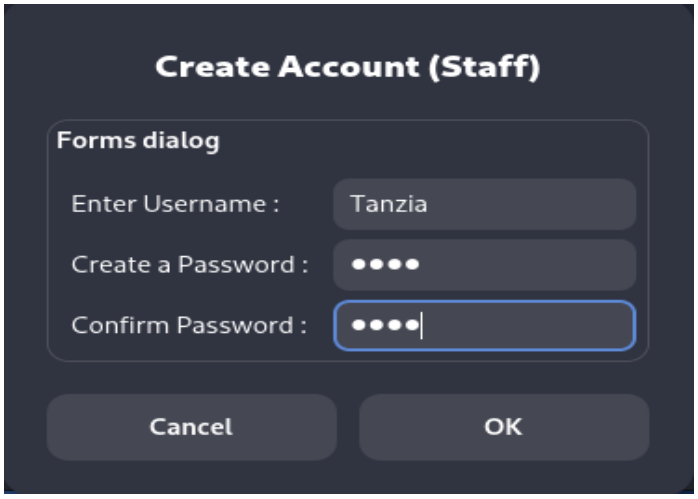


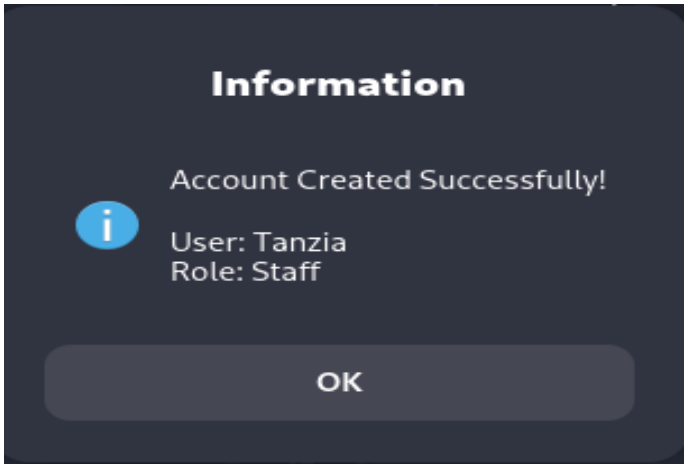

Create Account (Pharmacist)
Forms dialog
Enter Username :
Sinthia
Create a Password :
Confirm Password :
Cancel
OK


Information
Account Created Successfully!
User: Sinthia
Role: Pharmacist
OK

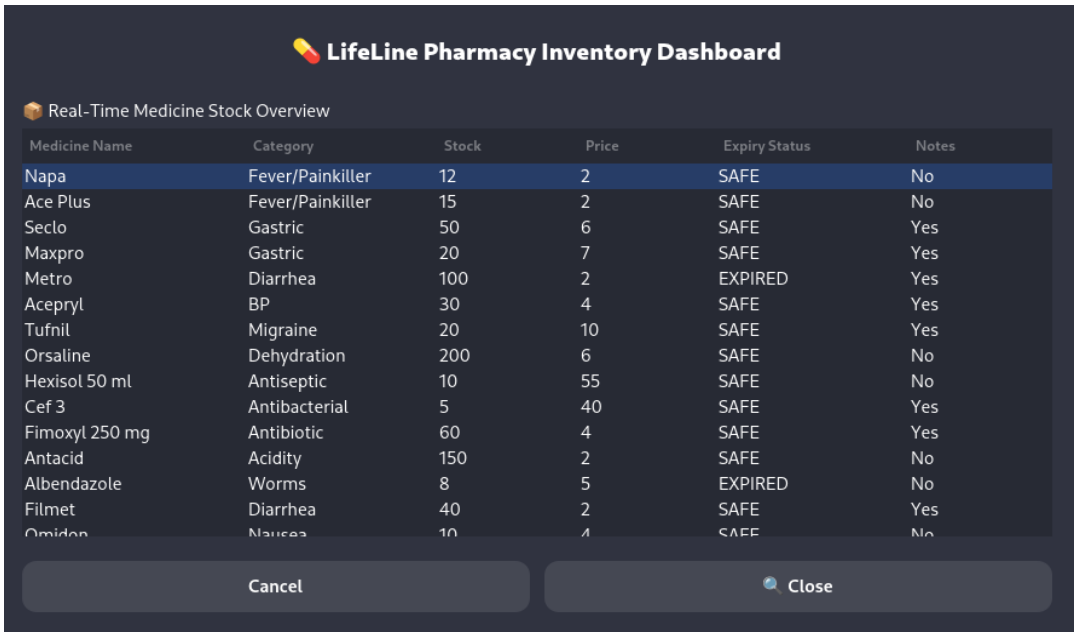
- **Add Staff:**


Select Account Type
Select items from the list below.
Role
Staff
Pharmacist
Cancel
OK



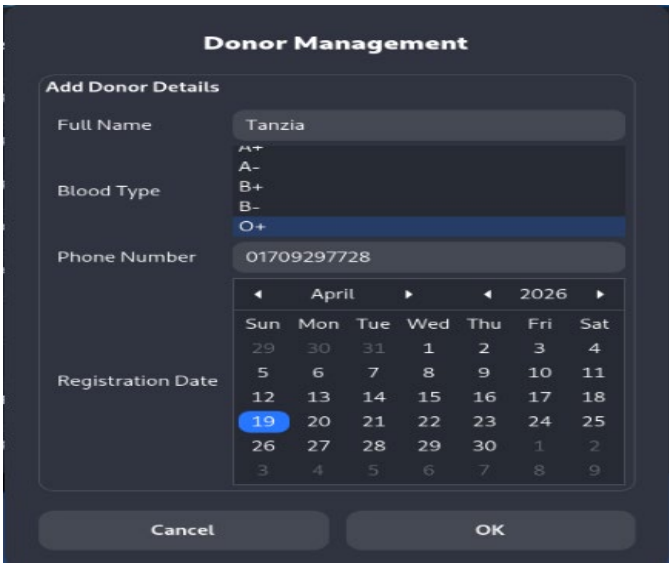


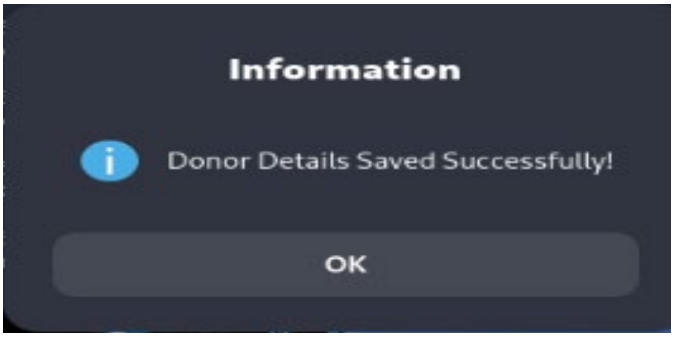
- **Medicine Inventory Dashboard:**



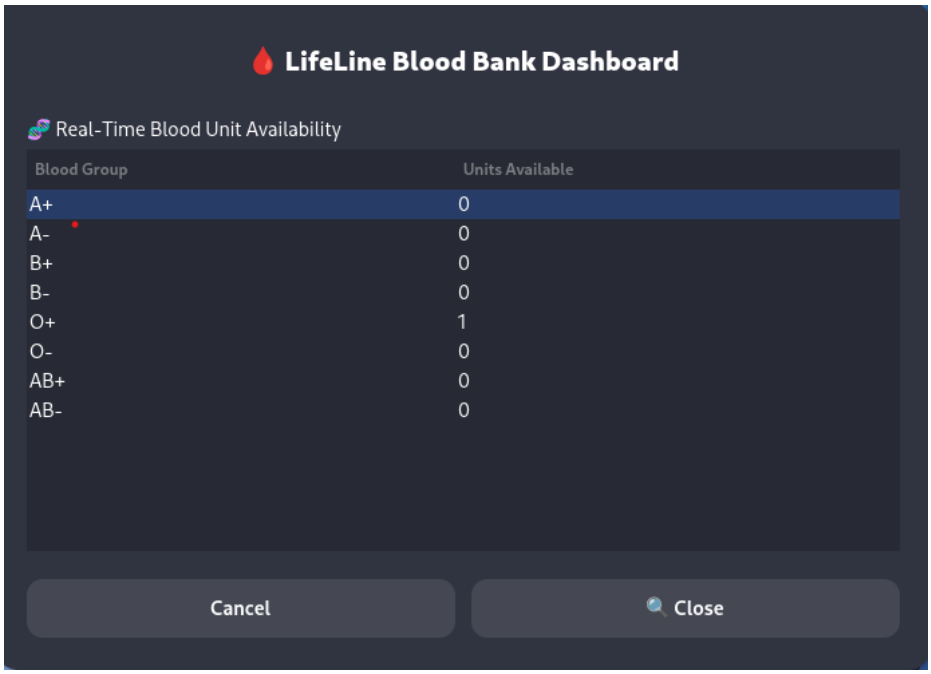
Medicine Name	Category	Stock	Price	Expiry Status	Notes
Napa	Fever/Painkiller	12	2	SAFE	No
Ace Plus	Fever/Painkiller	15	2	SAFE	No
Seclo	Gastric	50	6	SAFE	Yes
Maxpro	Gastric	20	7	SAFE	Yes
Metro	Diarrhea	100	2	EXPIRED	Yes
Acepryl	BP	30	4	SAFE	Yes
Tufnil	Migraine	20	10	SAFE	Yes
Orsaline	Dehydration	200	6	SAFE	No
Hexisol 50 ml	Antiseptic	10	55	SAFE	No
Cef 3	Antibacterial	5	40	SAFE	Yes
Fimoxyl 250 mg	Antibiotic	60	4	SAFE	Yes
Antacid	Acidity	150	2	SAFE	No
Albendazole	Worms	8	5	EXPIRED	No
Filmet	Diarrhea	40	2	SAFE	Yes
Omidon	Nausea	10	4	SAFE	No

- **Add Donor:**

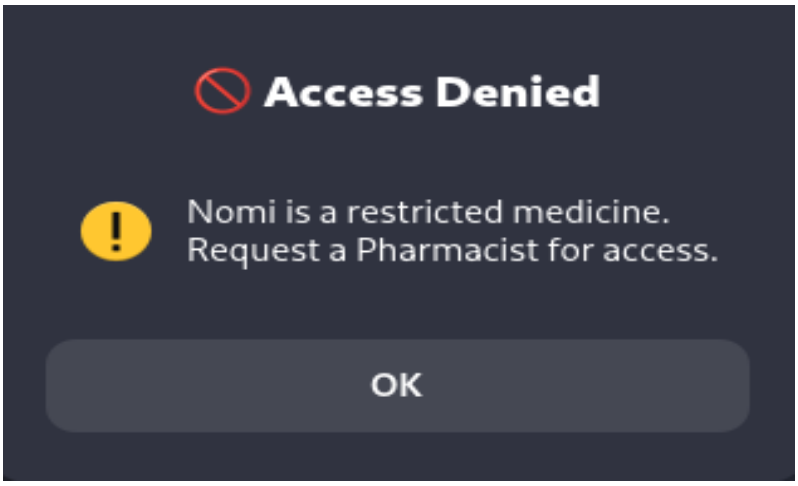




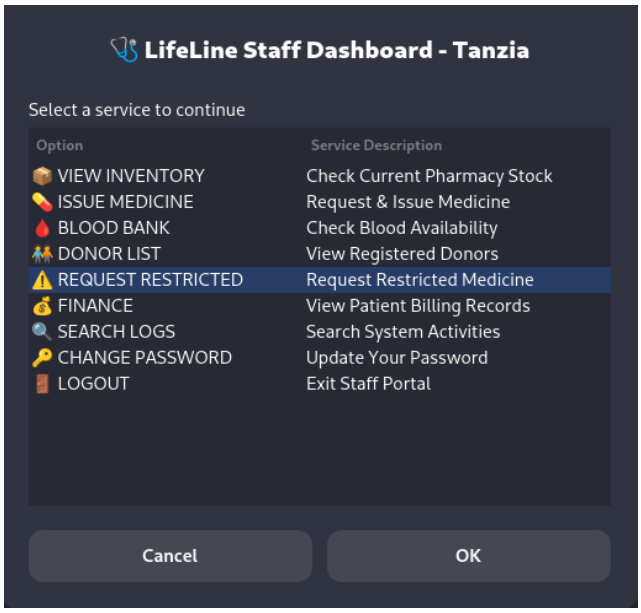
- **Real- Time Blood Bank Availability:**

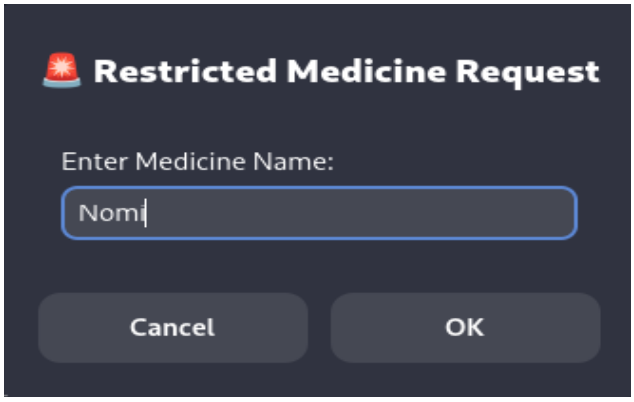


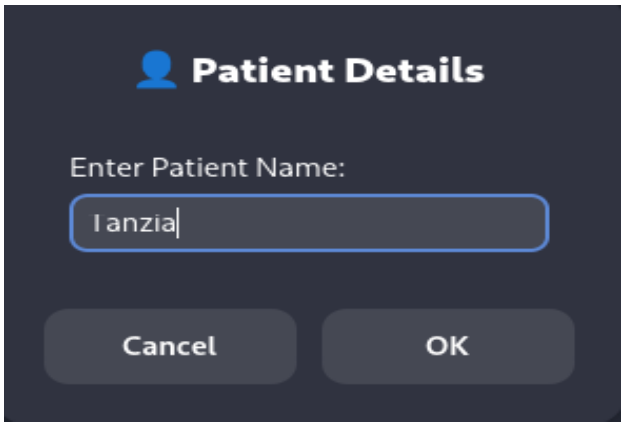
- **Request Restricted Medicine:**

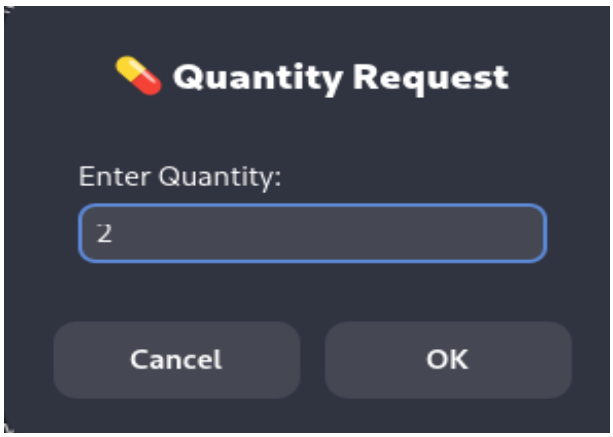


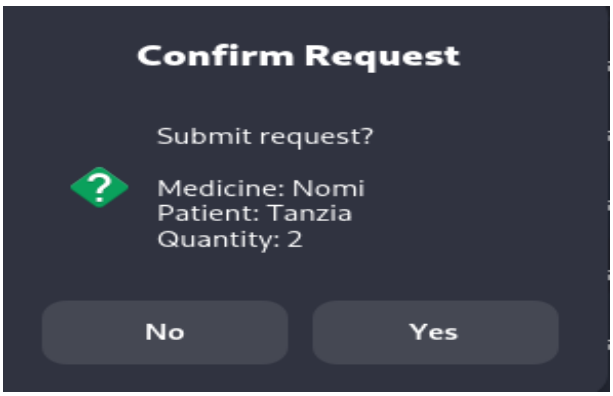
- **Request Restricted Medicine to Access:**

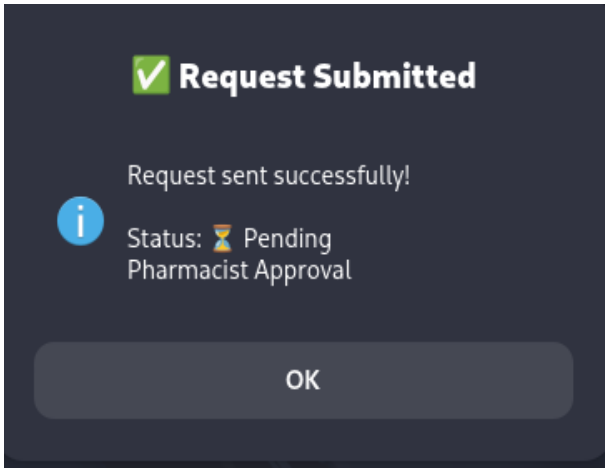




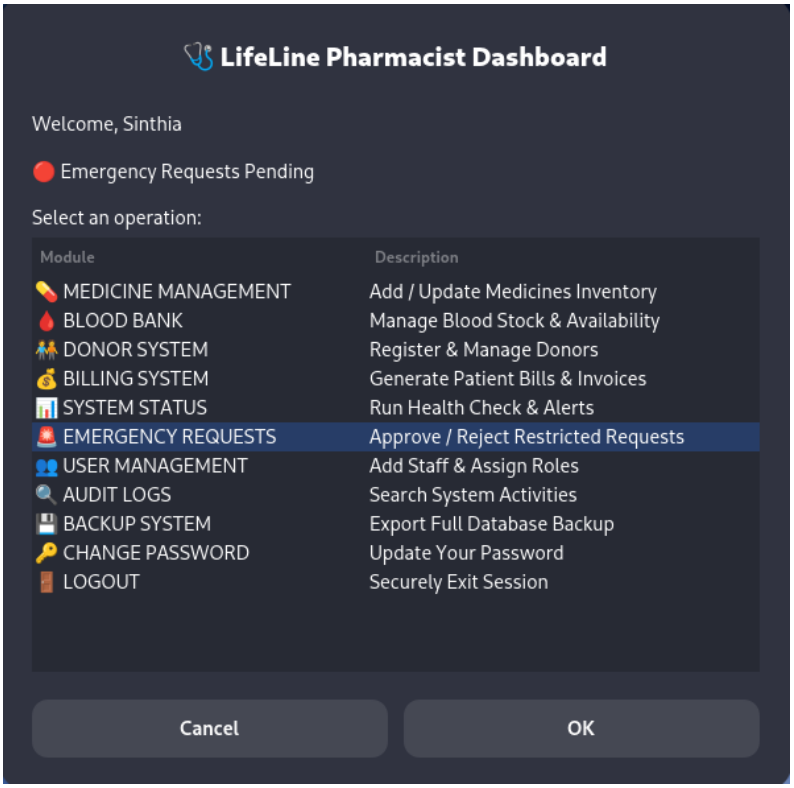

Patient Details
Enter Patient Name:
Tanzia
Cancel
OK


Quantity Request
Enter Quantity:
2
Cancel
OK

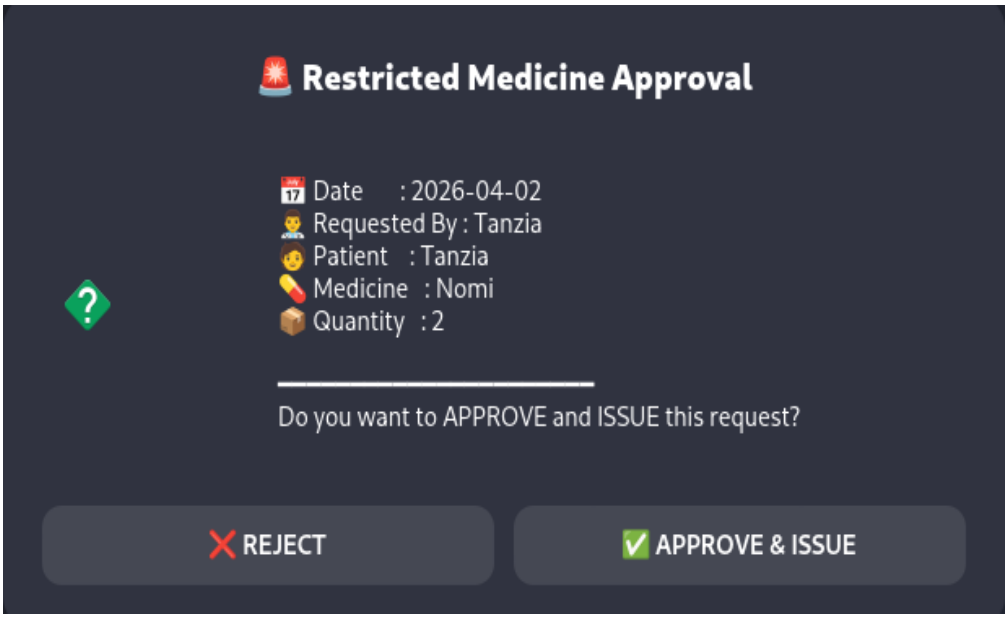

Confirm Request
Submit request?
Medicine: Nomi
Patient: Tanzia
Quantity: 2
No
Yes


Request Submitted
Request sent successfully!
Status: Pending
Pharmacist Approval
OK

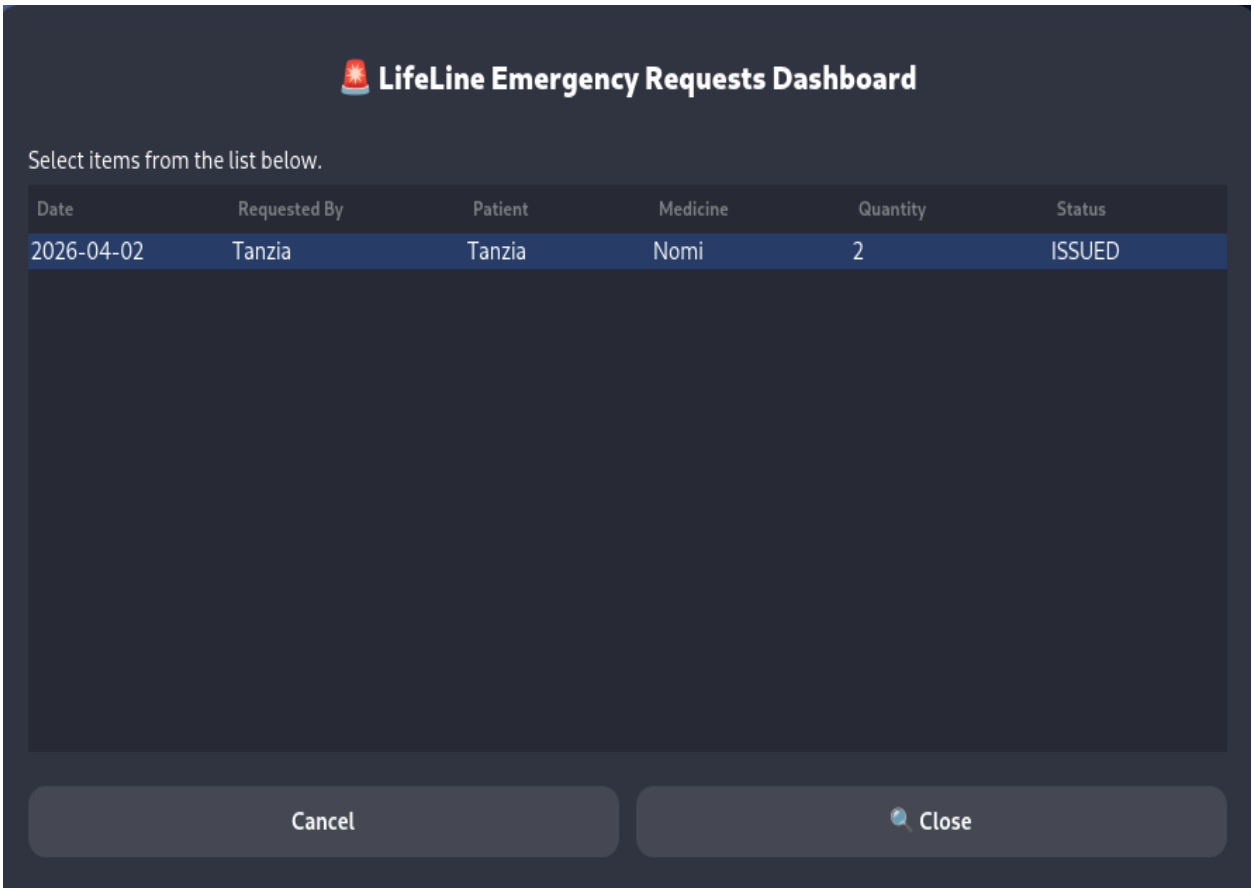
- **Emergency Requests Pending:**



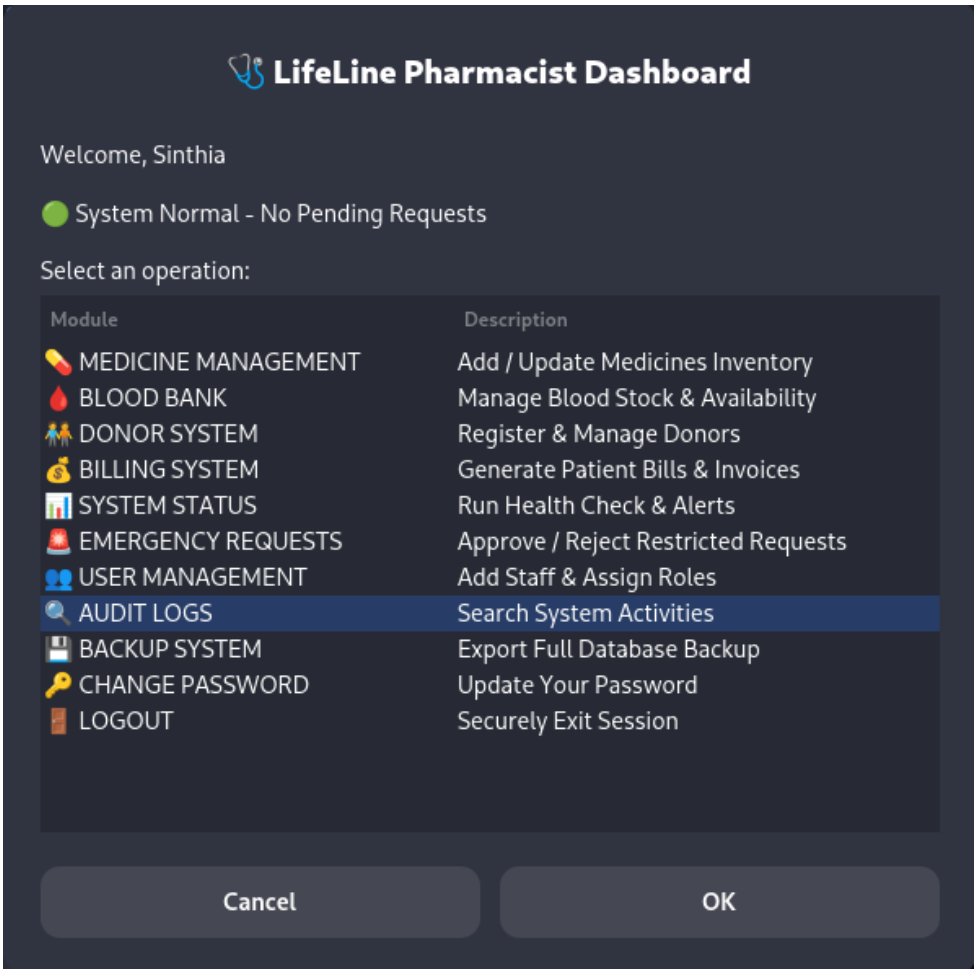
- **Approval of Restricted Medicine:**

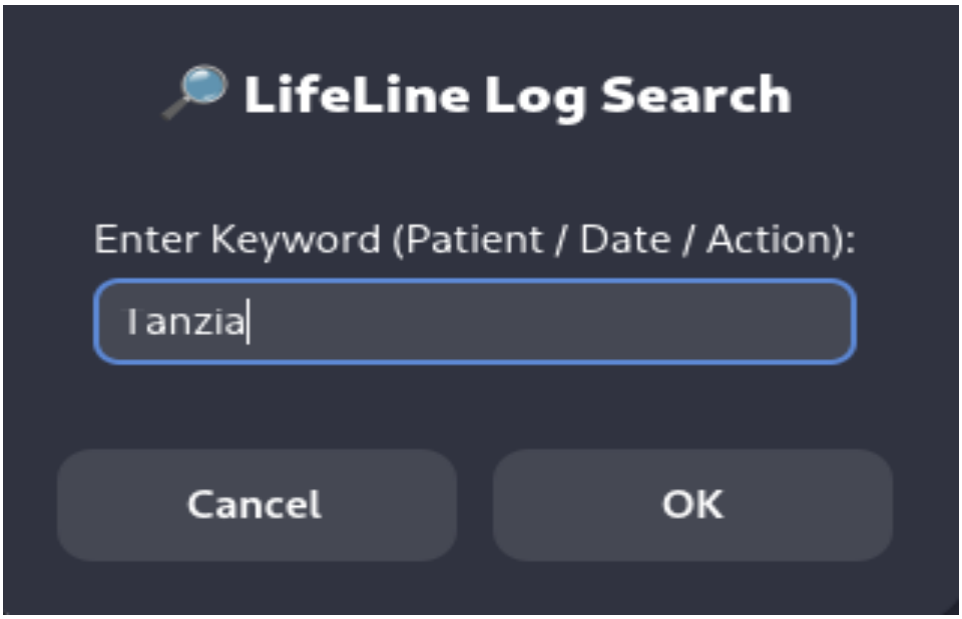


- **Emergency Request Dashboard:**



- **Audit Logs:**


LifeLine Pharmacist Dashboard
Welcome, Sinthia
System Normal - No Pending Requests
Select an operation:
Module
Description
MEDICINE MANAGEMENT
Add / Update Medicines Inventory
BLOOD BANK
Manage Blood Stock & Availability
DONOR SYSTEM
Register & Manage Donors
BILLING SYSTEM
Generate Patient Bills & Invoices
SYSTEM STATUS
Run Health Check & Alerts
EMERGENCY REQUESTS
Approve / Reject Restricted Requests
USER MANAGEMENT
Add Staff & Assign Roles
AUDIT LOGS
Search System Activities
BACKUP SYSTEM
Export Full Database Backup
CHANGE PASSWORD
Update Your Password
LOGOUT
Securely Exit Session
Cancel
OK


LifeLine Log Search
Enter Keyword (Patient / Date / Action):
Ianzia
Cancel
OK

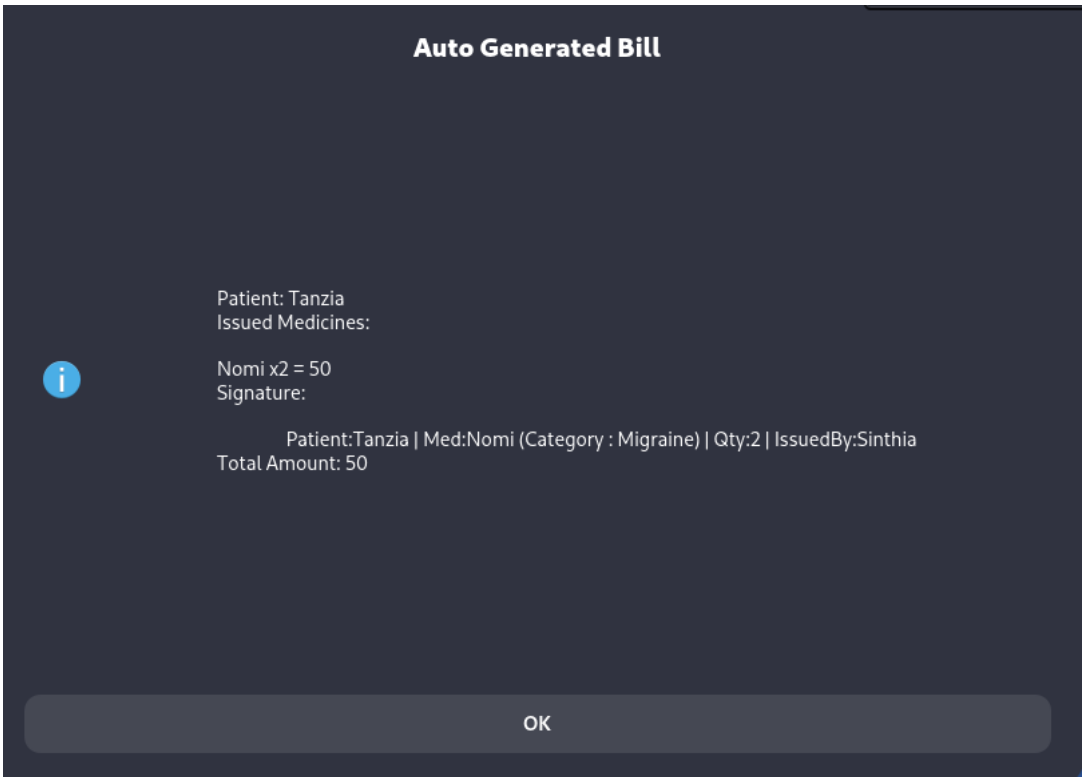
LifeLine Audit Logs

Showing results for: 'Tanzia'

Date	User	Action	Details
2026-04-09 12:19:21	User: Taufique	[AUTHENTICATION]	Failed Login Attempt : Tanzia
2026-04-09 12:19:54	User: Sinthia	[USER ADDITION]	New Staff Account Created: Tanzia
2026-04-09 12:21:27	User: Sinthia	[DONOR]	Registered Donor : Tanzia
2026-04-09 12:22:05	User: Tanzia	[AUTHENTICATION]	Successful Login
2026-04-09 12:23:45	User: Tanzia	[ISSUE]	Issued 1 units of Nutrum Gold (Multivitamin) to Patient: Sinthia
2026-04-09 12:24:35	User: Tanzia	[SECURITY]	Access Denied: Tufnil requested by Tanzia
2026-04-09 12:25:57	User: Tanzia	[AUTHENTICATION]	Successful Login
2026-04-09 12:27:04	User: Tanzia	[REQUEST]	Restricted medicine requested: Tufnil Qty:2 Patient:Hridy Status:DENIED
2026-04-09 12:28:39	User: Tanzia	[AUTHENTICATION]	Successful Login
2026-04-09 12:28:59	User: Tanzia	[REQUEST]	Restricted medicine requested: Tufnil Qty:1 Patient:Hridy Status:DENIED
2026-04-09 12:34:23	User: Tanzia	[AUTHENTICATION]	Successful Login

Cancel Close

- **Billing Invoice:**



Auto Generated Bill

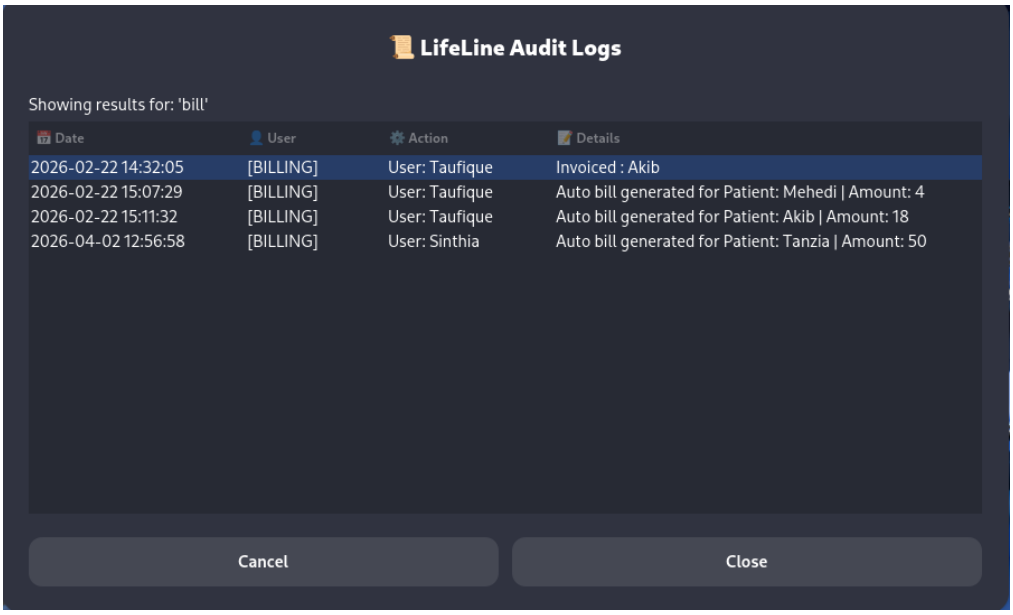
Patient: Tanzia
Issued Medicines:

Nomi x2 = 50
Signature:

Patient:Tanzia | Med:Nomi (Category : Migraine) | Qty:2 | IssuedBy:Sinthia
Total Amount: 50

OK

- **Billing History:**



3.3 System Performance Analysis

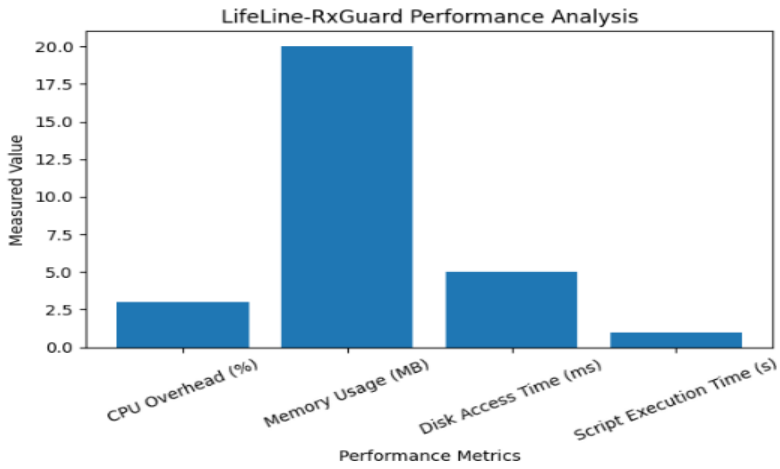
The performance of LifeLine-RxGuard was evaluated to ensure speed, efficiency, and stability, focusing on:

- **Script Response Speed** – Time taken for record retrieval, billing, and donor registration.
- **File Processing Efficiency** – Handling large datasets in medsInventory.txt.
- GUI Responsiveness – Smooth Zenity interface navigation and alerts.
- **Record Handling Stability** – Consistent performance during multiple operations.

Performance Testing

The *medsInventory.txt* file was populated with unique medicine records:

- **Search Latency**: Using grep, searches for specific medicines took less than 0.08 seconds.
- **Memory Footprint:** The application uses only 20 MB of RAM, making it far more efficient than web-based PMS tools.
- **Disk I/O:** Flat-file writes are atomic and near-instant, leveraging lightweight Linux utilities for fast processing.



LifeLine-RxGuard delivers **fast, stable, and resource-efficient performance**, combining Bash scripting, flat-file storage, and a Zenity GUI. It provides a practical alternative to conventional ERP systems, with minimal memory usage and rapid data handling.

3.4 Results & Discussion

The testing results indicate that LifeLine-RxGuard successfully performs system monitoring tasks using a **Zenity** based **GUI**.

The following observations were made during testing:

- Login system works correctly
- Pharmacist and staff roles operate separately
- Medicine inventory updates successfully
- Restricted medicine requests are controlled
- Billing records are generated correctly
- Blood inventory updates accurately
- Audit records preserve activity history

Overall, **LifeLine-RxGuard** effectively demonstrates how **Bash scripting combined with Zenity** can automate system monitoring tasks, providing an accurate, efficient, and visually interactive tool suitable for both novice and experienced Linux users.

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

LifeLine-RxGuard is a lightweight Linux monitoring and protection tool designed primarily for educational and practical system administration purposes.

The development of this tool contributes positively to the technological learning environment by providing a simple yet effective method for analyzing system performance and detecting potential issues in Linux systems.

The project demonstrates how open-source technologies can be used to build practical solutions for real-world problems without requiring expensive software or complex infrastructure.

4.1.1 Impact on Life

LifeLine-RxGuard directly improves **patient safety** by reducing the risk of dispensing expired medicines. The system automatically checks medicine expiry dates during inventory and billing operations, helping pharmacists identify unsafe drugs before they reach patients. This lowers human error and improves treatment reliability.

The **blood bank module** also has an important life-saving role. It allows staff to quickly search donor records and blood groups during emergencies, which helps provide blood faster for surgeries or critical situations.

4.1.2 Impact on Society & Environment

The project promotes Sustainable Computing. Most modern software forces hospitals to upgrade their hardware every 5 years. LifeLine-RxGuard can run on recycled hardware, reducing electronic waste. It also facilitates a paperless environment, saving thousands of sheets of paper per year for a standard clinic.

4.1.3 Ethical Aspects

Ethical engineering was maintained by ensuring Non-Repudiation. Every bill and inventory change is tagged with the specific User ID of the person who performed the action. This prevents fraud and ensures that staff are accountable for the medication they handle.

4.1.4 Sustainability Plan

LifeLine-RxGuard is designed for the long term. Because it relies on POSIX-standard Bash, the code written today will likely still run on Linux systems 20 years from now. It is easy to maintain because any IT student with basic Linux knowledge can read and modify the .sh file.

4.2 Project Management and Team Work

The project team followed a structured development process to successfully implement the **LifeLine-RxGuard Linux system monitoring tool**. The team used **GitHub for version control**, collaborative communication tools, and **weekly development milestones** to manage tasks and track project progress.

The project development process included requirement analysis, Bash script development, testing in the Linux environment, debugging, and documentation preparation. Each team member contributed to different tasks such as script implementation, testing system monitoring commands, improving script efficiency, and preparing the final project report.

Version control through GitHub helped maintain different script versions and ensured that all changes were documented properly. Structured communication among team members allowed efficient problem solving and smooth coordination throughout the project development lifecycle.

Regular review sessions and feedback cycles were also conducted to continuously improve the system's performance and reliability. This organized approach ensured timely completion of the project while maintaining high quality and consistency across all components.

Budget Analysis

The project required minimal financial resources because it was implemented using **open-source Linux tools and Bash scripting**.

• **Total Actual Cost:** 200 BDT (electricity consumption only)
• **Alternate Budget (Laptop Rent + Internet):** 700 BDT

Since the system runs directly in a Linux terminal environment, no additional commercial software or expensive infrastructure was required.

Educational Impact

• Helps students understand **operating system concepts** such as **process management, memory usage, and disk monitoring**.
• Provides practical experience with **Linux commands and Bash scripting**.
• Allows students to **observe real-time system resource usage** and understand system behavior.
• Encourages self-learning in **Linux system administration and shell scripting techniques**.

Economic Impact

• The system is based on **open-source technologies**, eliminating the need for expensive monitoring software.
• Requires **minimal hardware resources**, making it accessible for most computer systems.
• The lightweight nature of the tool reduces operational costs compared to large enterprise monitoring applications.

Social Impact

• Provides an easy-to-use monitoring tool for **students, beginners, and Linux users**.
• Encourages **digital literacy and technical learning** in operating systems and system administration.
• Makes system monitoring tools accessible to users who cannot afford expensive commercial software.

Sustainability Impact

- Lightweight and energy-efficient, requiring minimal hardware and low system resources.
- Paperless operation reduces waste through digital medicine, billing, and donor record management.
- Modular script design ensures long-term maintainability and future upgrades.
- Uses open-source Linux tools, making the system zero-cost and accessible for low-resource healthcare settings.
- Secure and reliable operation minimizes data loss and supports responsible resource use.

LifeLine-RxGuard Addresses Multiple CEP Criteria:

- **Multi-Stage Healthcare Workflow Integration**

The system processes multiple healthcare operations including medicine inventory, billing, blood stock, donor records, and restricted medicine approval through separate coordinated modules.

- **Automated Command Execution**

The system automatically executes Bash functions and file operations without requiring manual command handling by the user.

- **Role-Based Operational Control**

Different system functions are controlled according to pharmacist and staff roles.

- **Structured File-Based Data Management**

The project manages healthcare records through organized flat-file storage.

- **Multi-Module Integration**

Several Linux utilities and Bash functions are integrated into one complete pharmacy management solution.

- **System Stability and Error Handling**

Input validation and file checks prevent operational failure.

- **Engineering Judgment**

Designing module interactions, role permissions, and healthcare workflow logic required analytical decision-making.

- **Healthcare Data Coordination**

The project coordinates multiple healthcare records while maintaining operational consistency.

4.3 Complex Engineering Problem

The project addresses the complex engineering problem of building a **Multi-Role Relational System** without using a Database Management System (DBMS). Although LifeLine-RxGuard is lightweight, it solves several engineering problems related to healthcare system automation.

The main challenges addressed include:

- Integrating multiple pharmacy operations into one script
- Designing a role-based graphical healthcare interface
- Managing file consistency across modules
- Controlling restricted medicine approval
- Handling billing records accurately
- Maintaining blood stock updates
- Preventing data duplication
- Preserving audit history
- Supporting modular expansion
- Ensuring Linux compatibility
- Implementing secure user authentication without a centralized database
- Coordinating multiple flat-file records while maintaining data integrity across different modules

These tasks required knowledge of:

- Linux system architecture
- Shell scripting
- File processing
- Role-based design
- Healthcare workflow planning

4.3.1 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

PO	Justification
PO1	The project applies fundamental knowledge of operating systems, Linux commands, and Bash scripting to design a practical hospital-integrated pharmacy management system. Concepts such as file handling, process automation, and user authentication are utilized.
PO2	The project analyzes inventory management, billing, and donor/blood record handling using structured flat-file storage. Linux utilities are combined to solve real-world hospital pharmacy management problems.
PO3	The project involves designing and implementing a complete pharmacy management system (LifeLine-RxGuard) using Bash scripting. Multiple features such as medicine inventory, billing, restricted authorization, and audit logging are integrated into a single automated Zenity-based GUI application.

4.3.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving

EP1	EP2	EP3	EP4	EP5	EP6	EP7
Dept of Knowledge	Range of Conflicting Requirements	Depth of Analysis	Familiarity of Issues	Extent of Applicable Codes	Extent of Stakeholder Involvement	Inter-dependence

EP	Explanation
EP1	Designing and implementing LifeLine-RxGuard using Bash scripting and Linux commands to manage medicine inventory, billing, blood stock, donor records, and user authentication.
EP2	The system needed to balance simplicity and functionality, remaining lightweight while providing complete pharmacy management features.
EP3	The project analyzed pharmacy data such as medicine stock, sales, donor contributions, and audit logs, presenting results in a structured and accessible format via the Zenity GUI.
EP4	Team members familiarized themselves with Bash scripting, Zenity, Linux utilities, and structured file storage to successfully implement the project.
EP5	Standard Bash scripting conventions and Linux utilities were followed to ensure compatibility and portability across Linux systems.
EP6	Stakeholders, such as instructors and evaluators, provided requirements and feedback regarding system functionality, usability, and reporting.
EP7	The system depends on multiple Bash scripts and Linux utilities working together to manage inventory, generate bills, maintain donor and blood records, and log audit trails.

Key Problem-Solving Activities

- Implemented medicine inventory management with efficient add, search, update, and delete operations.
- Developed billing automation for accurate, fast transactions.
- Integrated restricted medicine authorization and audit logging for professional compliance.
- Managed blood inventory and donor records with structured flat-file storage.
- Tested the Zenity GUI to ensure user-friendly interaction for hospital staff.

4.3.3 Complex Engineering Activities

Table 4.3: Mapping with complex engineering activities

EA1	EA2	EA3	EA4	EA5
Range of resources	Level of Interaction	Innovation	Consequences for society and environment	Familiarity

EA	Explanation
EA1	Gathered pharmacy management requirements and identified **Bash scripting commands** and Zenity GUI elements needed to implement the system.
EA2	Prepared system workflow descriptions, user access control mechanisms, and technical documentation for LifeLine-RxGuard.
EA3	Developed **an automated Bash script integrating multiple modules** for inventory, billing, blood management, donor registration, and audit logging.
EA4	Conducted testing and troubleshooting to ensure efficient operation, minimal resource usage, and error-free execution.
EA5	Prepared detailed project documentation, including design, implementation, results, and future scope for evaluators.

Key Engineering Activities

- Designed modular **pharmacy management scripts** using Bash.
- Integrated **Zenity GUI** for improved user interaction.
- Implemented **inventory, billing, and blood/donor modules** in a single system.
- Optimized script execution for **low memory footprint (~20MB RAM)** and fast response times.
- Documented project workflow, testing results, and future development opportunities.

Chapter 5
Conclusion

5.1 Summary

The **LifeLine-RxGuard** project demonstrates the practical application of **Linux shell scripting** in developing a **fully functional, lightweight, and secure pharmacy management system**. Unlike traditional perceptions of shell scripts being limited to simple automation tasks, this project illustrates that, with a **modular design and structured workflow**, Bash scripting can effectively handle complex operations required in a real-world healthcare environment.

The system integrates multiple pharmacy operations—including **medicine inventory, billing, donor management, blood inventory, and audit logging**—under **role-based control**, ensuring both efficiency and security. The **Zenity-based graphical interface** enhances usability, transforming command-line operations into **intuitive, menu-driven interactions**, suitable for hospital staff with minimal technical knowledge.

A major highlight of LifeLine-RxGuard is its **lightweight and portable architecture**. The system requires **no SQL servers, external frameworks, or web deployment**, making it **easy to set up, resource-efficient, and suitable for environments with limited infrastructure**. The **Zenity-based graphical interface** enhances user experience by converting command-line operations into **intuitive, menu-driven interactions**, making it accessible to healthcare staff without programming knowledge.

The project also emphasizes **practical real-world controls**, such as **restricted medicine authorization** and **audit logs**, which are essential for professional pharmacy operations and compliance. Furthermore, the modular design ensures that the system can be **expanded in the future**, for instance by integrating databases, barcode scanning, web interfaces, multi-terminal networking, and mobile accessibility.

Overall, **LifeLine-RxGuard achieves all its intended objectives**. It demonstrates that a **hospital-integrated pharmacy management system can be both powerful and lightweight**, delivering security, efficiency, and usability. The project transforms operating system concepts and scripting knowledge into a **real-world healthcare solution**, bridging the gap between educational programming projects and professional pharmacy management tools.

By proving that **Linux shell scripting can support full-scale operational workflows**, LifeLine-RxGuard serves as a **robust, educational, and practical example** of how simple technologies can be leveraged to build effective healthcare management systems. It also highlights the potential of open-source tools in developing cost-effective digital healthcare solutions. The project provides a strong foundation for future research and advanced healthcare automation using lightweight Linux-based technologies.

Key Achievements of the Project

- **Comprehensive Pharmacy Management**
  - Integrated multiple pharmacy operations into a single system, including medicine inventory, billing, donor management, and blood inventory.
- **Lightweight Architecture**
  - Developed entirely with Bash shell scripting and flat-file storage, requiring no SQL servers, external frameworks, or web deployment.
  - Low memory footprint (~20MB RAM) ensures high efficiency and portability.
- **Role-Based User Authentication**
  - Implemented secure access control, allowing different users (pharmacists, staff, administrators) to perform tasks according to their roles.
- **Restricted Medicine Authorization**
  - Added pharmacist approval for dispensing restricted medicines, improving safety and compliance.
- **Automated Billing System**
  - Generates accurate pharmacy bills automatically, reducing human errors and saving time.
- **Audit Logging**
  - Maintains detailed logs of all actions, providing accountability and traceability for **professional operations.**
- **Zenity-Based GUI**
  - Converts terminal commands into a menu-driven graphical interface, enhancing usability for healthcare staff.
- **Efficient Data Handling**
  - Supports fast search, stable record handling, and low disk I/O even with large datasets.
- **Blood and Donor Management**
  - Tracks blood inventory and donor information systematically, enabling better hospital resource management.
- **Future-Ready Modular Design**
  - Modular architecture allows future enhancements like barcode scanning, PDF bill generation, web/mobile interfaces, multi-terminal access, and database integration.

- **Educational and Practical Value**
    - Demonstrates how Linux shell scripting and system concepts can be applied to real-world healthcare management, combining learning with practical implementation.

5.2 Limitations

Although **LifeLine-RxGuard** performs successfully as a lightweight pharmacy management system, the current version has several limitations due to its design for **local, single-terminal execution**:

- **Single-Terminal Environment**:
  The system cannot natively sync data between multiple computers. Network-based multi-terminal usage requires external shared-drive configurations.
- **Flat-File Storage**:
  All records are stored in text files. Very large data volumes may reduce efficiency and slow down search and processing operations over time.
- **Local Execution Dependency**:
  LifeLine-RxGuard currently runs only on one Linux machine and does not support multi-terminal synchronization or remote access.
- **Limited Graphical Reporting**:
  The Zenity interface does not support high-end graphical reports such as bar charts, limiting visual analytics capabilities.
- **Missing Integrations**:
    - No barcode scanner integration for faster medicine issue
    - No automatic prescription import
    - No online backup support
    - No printable invoice or PDF bill format
    - No advanced search indexing for rapid queries
- **Basic User Authentication**:
  Credentials are stored in local text files with lightweight hashing. This provides functionality but is less secure compared to encrypted database storage.
- **Dependency on Zenity**:
  The GUI requires Zenity to be installed on Linux systems. Systems without Zenity must install it first to use the interface.

These limitations do **not prevent the system from functioning**, but they define the **current boundaries** and highlight potential areas for future enhancement.

5.3 Future Work

The **LifeLine-RxGuard** project has significant potential for future expansion to enhance scalability, usability, and functionality. Possible improvements include:

- **Database Upgrade**:
  Future versions may replace the current flat-file storage with **relational databases** such as **MySQL** or **PostgreSQL**, improving **data scalability, search efficiency, and multi-user support**.
- **Barcode Integration**:
  Adding **barcode scanning** can make medicine issuance **faster and more accurate**, reducing manual entry errors. Future enhancements may include:
  - Barcode-based medicine entry
  - Hardware scanner integration for real-time billing
- **Automated Reporting and Notifications**:
  - **PDF bill generation** for easier record-keeping
  - **Automatic expiry notifications** for medicines and blood stock
  - **Printable transaction reports** for audit and management purposes
- **Network and Multi-User Support**:
  Implementing **network-based access** using tools like *sshfs* would allow **multiple pharmacy terminals to share a central data folder**, enabling real-time inventory updates across departments.
- **Security Enhancements**:
  Integrating **OpenSSL encryption** would secure database files at rest, enhancing data protection.
- **Web and Mobile Interfaces**:
  - A **web interface** could enable browser-based access for easier management.
  - **Mobile compatibility** would allow **emergency donor lookup** and on-the-go management.
- **Backup and Automation**:
  - Scheduled backups to ensure data safety
  - Automated routines for routine inventory checks and audit logs

With the current **modular architecture**, these upgrades are **technically achievable**, allowing LifeLine-RxGuard to evolve into a fully-featured, secure, and scalable pharmacy management system.

Final Remark

LifeLine-RxGuard successfully demonstrates that even lightweight, local Linux-based systems can provide robust, secure, and practical solutions for hospital pharmacy management. By combining modular Bash scripting, flat-file storage, and a **Zenity** GUI, the project bridges the gap between educational programming concepts and real-world healthcare applications. Its efficient

performance, role-based controls, and future-ready design make it a powerful yet simple tool, proving that innovation does not always require complex or expensive technologies.

References

- Linux Documentation Project
- GNU Bash Documentation*: The Definitive Guide to Shell Scripting.*
- Zenity Project: *GTK+ Dialogs for Shell Scripts.*
- Shotts, W. (2019). *The Linux Command Line: A Complete Introduction.*
- *Linux System Administration Guide*
- *GitHub Repository – LifeLine-RxGuard*